



Capítulo 4 - Controle de vários desvios

Cada desvio precisa de identificação, saída de controle e ajuste próprios.

Introdução	No Curso Básico, comandamos um desvio. Agora vamos organizar o controle de três desvios, aproveitando funções, decisões e repetições.
Objetivo	Identificar e comandar vários desvios por uma ESP32, mantendo os pinos e os ângulos de cada servo separados.
Meta de aprendizagem	Ao final, você deverá conseguir relacionar cada desvio a um servo e a um GPIO, enviar comandos individuais e registrar seus ajustes.

1. Um controle para cada desvio

Cada servomotor recebe seu próprio sinal de controle. Assim, o programa pode alterar um desvio sem mandar os demais para a mesma posição.

O servo recebe comandos por pulsos elétricos. A biblioteca ESP32Servo gera esses pulsos, permitindo informar a posição com `write()`. Cada servo será representado por um objeto Servo.

O número do desvio não é o número do GPIO. D6 identifica um desvio da ferrovia; GPIO 19 identifica uma saída da ESP32.

2. Organização do exemplo

Utilizaremos D5, D6 e D8 como exemplos de desvios da gaveta GA3. O quadro abaixo é um mapeamento didático de bancada: não substitui as ligações definitivas da gaveta GA3.

Desvio	Índice no programa	GPIO de exemplo
D5	0	18
D6	1	19
D8	2	21

Esses GPIOs são exemplos para uma placa ESP32 clássica baseada no ESP32-WROOM, com os pinos disponíveis. Outros modelos de ESP32 podem exigir outro mapeamento.

O exercício começa em simulação, apenas pelo Monitor Serial. Depois, o mesmo código poderá ser utilizado com três servos de bancada, após conferir as ligações e os ajustes.



3. Guardando os dados em listas

Uma lista de valores do mesmo tipo é chamada de array. Ela permite guardar, em conjunto, os números dos desvios, os pinos e os ângulos.

```
const int turnoutIds[3] = {5, 6, 8};  
const int servoPins[3] = {18, 19, 21};  
const int straightAngles[3] = {67, 67, 67};  
const int divergingAngles[3] = {97, 97, 97};
```

O primeiro elemento tem índice 0, o segundo tem índice 1 e o terceiro tem índice 2. Por exemplo, servoPins[1] vale 19 e pertence ao D6.

As listas seguem a mesma ordem. straightAngles guarda os ângulos de DI e divergingAngles guarda os ângulos de AC.

Os valores 67 e 97 são exemplos. Antes de conectar a haste a um desvio, ajuste individualmente os dois valores. Use o TTD-01 para encontrar o curso adequado, com encosto suave das agulhas, e registre os resultados.

4. Uma função para escolher o desvio e a posição

A função setTurnout() receberá dois parâmetros: o índice do desvio e a posição desejada. O valor true representa AC; false representa DI.

```
setTurnout(0, false); // D5: DI  
setTurnout(1, true);  // D6: AC  
setTurnout(2, false); // D8: DI
```

A função procura o ângulo na lista correspondente e comanda somente o servo escolhido. No modo de simulação, ela apenas mostra a mensagem.

O código verifica o índice antes de acessar as listas. Um índice negativo ou maior que 2 não corresponde a nenhum desvio deste exemplo.

5. Comandos pelo Monitor Serial

O programa terá seis comandos. Digite um caractere e envie pelo Monitor Serial:



Comando	Ação
1	D5 para DI
2	D6 para DI
3	D8 para DI
4	D5 para AC
5	D6 para AC
6	D8 para AC

O programa ignora os caracteres de final de linha e informa quando recebe outro comando inválido. Para este exercício, envie apenas um comando por vez.

6. Programa completo

Na Arduino IDE, instale a biblioteca ESP32Servo pelo gerenciador de bibliotecas. Crie um programa chamado `intermediate_04_turnouts` e copie o código abaixo.

O valor `simulationOnly = true` mantém os GPIOs dos servos sem inicialização pela biblioteca. Dessa forma, o primeiro teste requer somente a ESP32 de bancada e o cabo USB.

```
#include <ESP32Servo.h>

const bool simulationOnly = true;
const int turnoutCount = 3;
const int turnoutIds[turnoutCount] = {5, 6, 8};
const int servoPins[turnoutCount] = {18, 19, 21};
const int straightAngles[turnoutCount] = {67, 67, 67};
const int divergingAngles[turnoutCount] = {97, 97, 97};
Servo turnouts[turnoutCount];

void setTurnout(int index, bool diverging) {
    if (index < 0 || index >= turnoutCount) {
        Serial.println("Invalid turnout index");
        return;
    }
    int angle = diverging ? divergingAngles[index]
                        : straightAngles[index];
    if (!simulationOnly) {
        turnouts[index].write(angle);
    }
    Serial.print("D");
    Serial.print(turnoutIds[index]);
    Serial.print(diverging ? " - AC - Angle: " : " - DI - Angle: ");
    Serial.println(angle);
}
```



Programa completo - continuação

```
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println(simulationOnly ? "Simulation mode" : "Servo mode");
  for (int index = 0; index < turnoutCount; index++) {
    if (!simulationOnly) {
      turnouts[index].setPeriodHertz(50);
      turnouts[index].attach(servoPins[index]);
    }
    setTurnout(index, false);
    if (!simulationOnly) delay(1000);
  }
  Serial.println("DI: 1, 2, 3 | AC: 4, 5, 6");
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command >= '1' && command <= '3') {
      setTurnout(command - '1', false);
    } else if (command >= '4' && command <= '6') {
      setTurnout(command - '4', true);
    } else if (command != '\n' && command != '\r') {
      Serial.println("Invalid command");
    }
  }
}
```

7. Entendendo o código

Servo turnouts[turnoutCount] cria uma lista com três objetos de controle. Cada um é associado a um GPIO por attach(), somente quando a simulação é desativada.

setPeriodHertz(50) configura os pulsos de controle para 50 Hz. Essa frequência é do sinal que controla o servo e não representa a velocidade de deslocamento das agulhas.

A expressão condição ? valorA : valorB é uma escolha compacta: utiliza o primeiro valor quando a condição é verdadeira e o segundo quando é falsa. No código, ela seleciona o ângulo de AC ou DI.

Serial.available() verifica se há caracteres recebidos. Serial.read() lê um caractere. As aspas simples, como em '1', representam um caractere.

A subtração command - '1' transforma os caracteres 1, 2 e 3 nos índices 0, 1 e 2. Para os comandos de AC, command - '4' produz os mesmos índices.

As mensagens indicam a posição comandada. O programa não confirma mecanicamente a posição das agulhas. Não utilizamos sensores Hall nos desvios neste exercício.

Exercício prático - Simulação

Meta: selecionar três desvios e conferir seus comandos sem movimentar a maquete.

1. Mantenha simulationOnly = true.



2. Selecione a placa e a porta, compile e envie o programa para uma ESP32 de bancada.
3. Abra o Monitor Serial em 115200 baud e pressione EN ou RESET.
4. Confira a mensagem de simulação e os três comandos iniciais de DI.
5. Envie o comando 5. Confira que somente o D6 recebe o comando AC.
6. Envie o comando 2. Confira o retorno do D6 para DI.
7. Teste os outros quatro comandos, um de cada vez.
8. Envie 9 e confira a mensagem de comando inválido.

Resultado inicial esperado:

```
Simulation mode
D5 - DI - Angle: 67
D6 - DI - Angle: 67
D8 - DI - Angle: 67
DI: 1, 2, 3 | AC: 4, 5, 6
```

Ao enviar 5 e depois 2:

```
D6 - AC - Angle: 97
D6 - DI - Angle: 67
```

Como desafio, altere apenas o segundo valor de cada lista de ângulos e observe a mudança nas mensagens do D6. Depois retorne aos valores que serão utilizados na calibração real.



8. Ligações para o teste com servos

Após concluir a simulação, prepare uma montagem separada de bancada. Utilize três servos, uma fonte regulada de 5 V com capacidade para a carga e uma ESP32 alimentada por USB.

Ligação	Destino
Positivo dos três servos	Positivo da fonte de 5 V.
GND dos três servos	Negativo da fonte de 5 V.
GND da ESP32	Negativo da mesma fonte, formando a referência comum.
Sinal do servo de D5	GPIO 18, neste exemplo.
Sinal do servo de D6	GPIO 19, neste exemplo.
Sinal do servo de D8	GPIO 21, neste exemplo.

Não alimente os servos pelo pino 3V3 nem pela saída USB da ESP32. Nesta montagem, o positivo da fonte dos servos não será ligado ao pino 5 V da placa. Una somente os GNDs e os sinais conforme o quadro.

Faça as conexões com todas as alimentações desligadas. Confira a identificação dos fios de cada servo e a polaridade da fonte antes de energizar.

9. Teste físico e ajuste individual

1. Inicie com os servos sem ligação mecânica aos desvios. Ao inicializar o controle, o eixo pode se movimentar.
2. Confira os GPIOs e ajuste as listas com os valores obtidos individualmente no TTD-01. Mantenha o mesmo ajuste de pulsos usado na calibração.
3. Troque `simulationOnly` para `false`, compile e envie.
4. Observe os três comandos iniciais para DI, enviados com intervalo de um segundo.
5. Envie um comando, aguarde o término do movimento e confira qual servo respondeu.
6. Teste DI e AC de cada servo separadamente.
7. Depois de conferir os movimentos, conecte as hastes com a alimentação desligada e faça o ajuste fino das agulhas.
8. Registre os GPIOs, os ângulos finais e qualquer alteração realizada.



Se o servo vibrar continuamente ou forçar o mecanismo, interrompa a alimentação e revise o curso e a montagem. Não aumente o ângulo para vencer uma resistência mecânica.

O intervalo inicial de um segundo é apenas uma pausa de bancada, não uma confirmação de que o movimento terminou. Vários servos podem continuar consumindo corrente para manter a posição. Este programa não controla os relés de alimentação da gaveta GA3 nem executa a lógica definitiva de segurança.

10. Próximos passos da automação

O exercício permite comandar vários desvios individualmente. Uma rota completa exigirá verificar a ocupação dos trechos, a combinação dos desvios e as condições de operação antes de enviar comandos.

Nos próximos capítulos, ampliaremos a leitura de sensores e as verificações necessárias. A organização do programa em módulos poderá separar o controle dos desvios das mensagens e das decisões de rota.

Resumo do capítulo

- Cada servo precisa de uma saída de sinal própria.
- O número do desvio e o número do GPIO são identificações diferentes.
- Arrays mantêm os dados de cada desvio na mesma ordem.
- Uma função pode receber o índice do desvio e a posição desejada.
- DI e AC exigem calibração individual.
- O teste começa em simulação e depois avança para a bancada.
- Uma mensagem de comando não confirma a posição mecânica.
- Controlar vários desvios é uma etapa anterior à definição de rotas seguras.

Próximo capítulo: Leitura de vários sensores.

Referência técnica

Biblioteca ESP32Servo: documentação e exemplos oficiais em <https://github.com/madhephaestus/ESP32Servo>